

AN OUTSIDE ANALYSIS · YOUTUBE MUSIC

Throne Empty.

What's actually going on under the hood — and why a music app built on a video platform can't ever quite feel like itself.

SUBJECT

YouTube Music · iOS & Web

DOCUMENT

Outside analysis

STRUCTURE

§ 01 – 05 · Six symptoms · Three figures

COMPANION

throne-empty.vercel.app

PREAMBLE

Before we start.

A read, not a reveal. A guess at causes from the shape of the symptoms.

A small disclaimer before anything else. I don't work at Google, I haven't seen YouTube Music's source code, and nobody on the outside actually has.

So nothing in this document is a claim about what is literally happening inside the app. What it *is*, is a read — based on user reports piling up across forums, the kind of patterns engineers recognize when they look at how something behaves from the outside, and a guess at what kinds of architectural choices would produce those exact symptoms.

The whole document follows one rhythm. **Here's what users are hitting. Here's the kind of problem that produces this symptom. Here's why that matters.**

ON METHOD

The figures in this document are structural illustrations, not architectural diagrams. They show patterns the symptoms imply. They are not blueprints of any real system. Treat the boxes the way you'd treat shapes in an X-ray: the bones aren't drawn to scale, but the fracture is in the right place.

An audio app living inside a video platform.

You can't make sense of YouTube Music's problems without understanding this one thing first.

YouTube Music isn't a music app that happens to be owned by Google. It's a music product layered on top of YouTube's infrastructure.

The content system underneath is the same one that serves the videos. The recommendation pipeline is shared. The search index is shared. When you search for a song in YT Music, you aren't querying a music-first search service — you're querying YouTube's video index with some music-flavored ranking on top.

This sounds like a small thing. It isn't. When the foundation is built for one content type (video) and you're trying to serve a fundamentally different listening behavior (audio), the seams show everywhere.

Audio listening is long-session, background-friendly, queue-driven, library-heavy. Video viewing is short-session, foreground-only, one-thing-at-a-time, discovery-driven. The data model, the player state, the navigation, the search results — all of it gets pulled in two directions, and one of those directions is the foundation.

Every symptom in this document is downstream of this one fact.

FIGURE 01

Two foundations.

A music product, built on what it needs — and a music product, built on what was already there.

FIG. 01 Service topology, side by side

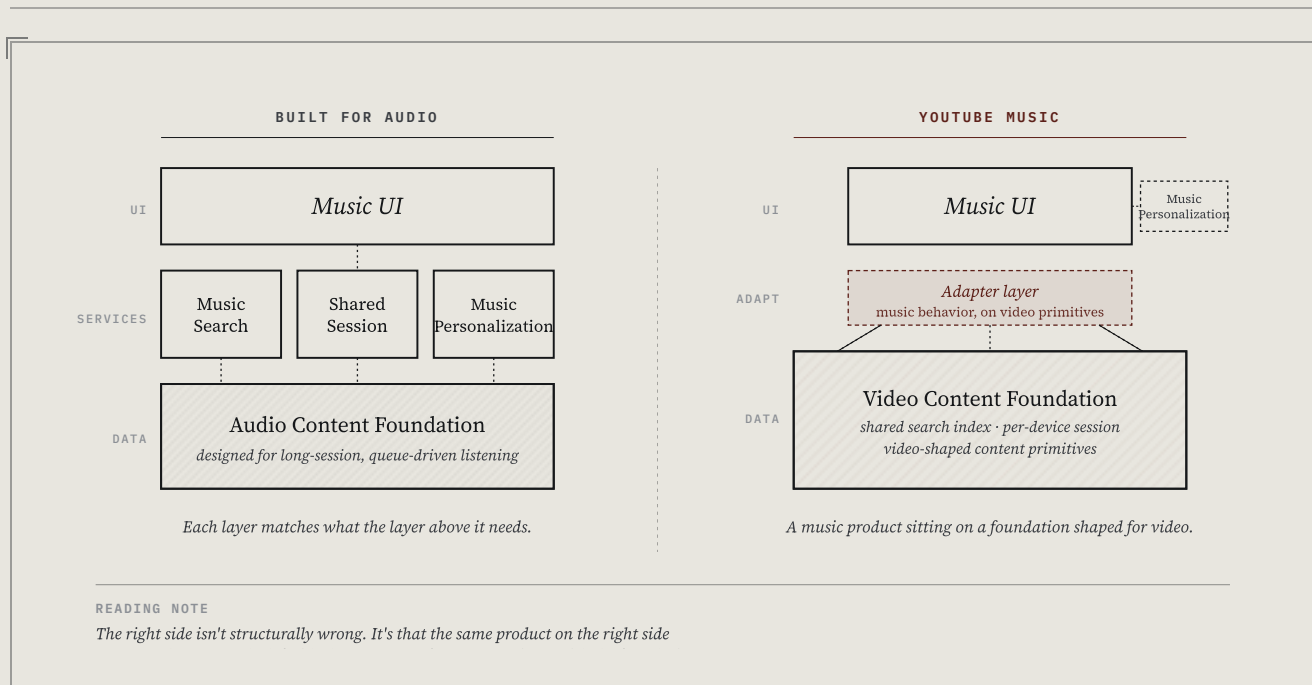


FIGURE NOTE. The left stack is what an audio-first product would be built on. The right stack is the shape YouTube Music ends up taking — adapters bridging a music UI to a foundation designed for video.

Six things that break, and why.

None of them are catastrophic alone. All of them point the same way.

What follows are six specific things people hit. The interesting part isn't that YouTube Music breaks in these specific ways — Spotify has scroll lag too, Apple Music has its own quirks. It's that all the breaks point in the same direction.

2.1 Rotation, removed.

WHAT USERS HIT

YT Music once supported landscape orientation. Now it doesn't. People have been asking on Google's own forums for years why the option was taken away. The answer is that the option no longer exists.

WHAT THIS TELLS ME defining a proper landscape layout means maintaining two layout trees, one per orientation, with state plumbing between them. If portrait was inherited from a system that wasn't designed with orientation as a first-class concept, you have two choices — invest the engineering effort to design landscape from scratch, or remove the option and pretend it was never there. They chose the second.

It's a small thing. But it's the kind of small thing you only do when the cost of doing it right is bigger than you're willing to pay.

2.2 The Now Playing screen has many faces.

WHAT USERS HIT

The Now Playing screen looks slightly different depending on how you got there. From the mini-player, one thing. After autoplay kicks in, slightly different. From a lock-screen notification, different again. Different controls surface in different orders. The same actions live in different places.

WHAT THIS TELLS ME a Now Playing screen should be one canonical view, driven by one canonical state model. Whatever is playing, however the user got there, the screen should render from the same template fed by the same data.

When the screen has variants depending on entry point, what you're usually looking at is multiple instantiations of the same surface, each one slightly customized for the path the user took. That's a thing you do when there isn't a clean shared view model — you patch each path independently until they all sort of work.

The cost compounds. Every time the design changes, three different versions need to be updated, and they always drift.

FIGURE 02

One surface, four faces.

The drift is the tell.

FIG. 02 *Four entry points — current state vs. canonical view model*

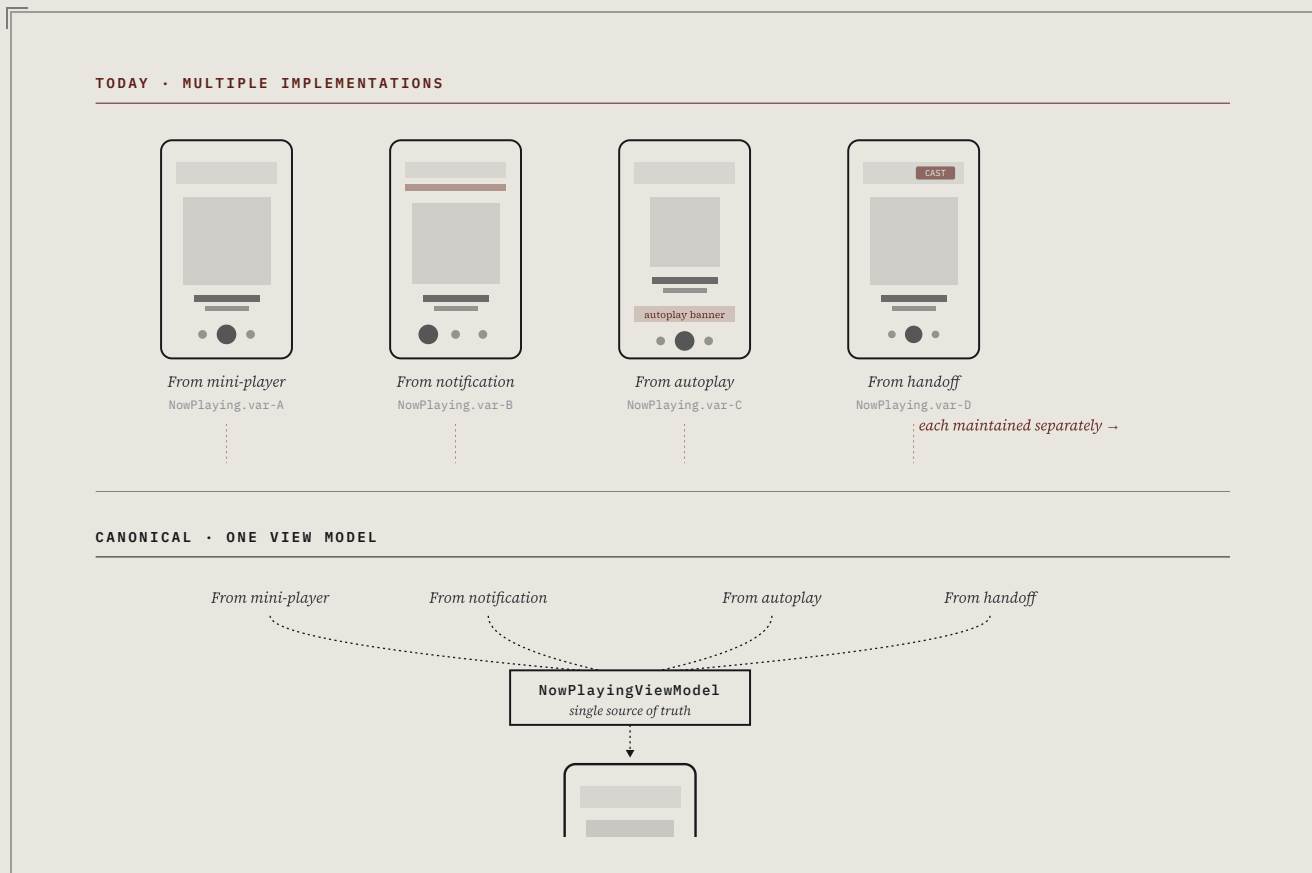


FIGURE NOTE. *When the same surface drifts depending on how the user reached it, what you're usually looking at is multiple implementations of one screen. The fix is structural: a single canonical view model.*

2.3 The library wall.

WHAT USERS HIT

Libraries with a few thousand songs cause real scroll lag. Multiple bug reports describe the app slowing past around a thousand items. The web version reproduces it. People with very large libraries report system slowdowns far beyond what a music app should ever cause.

WHAT THIS TELLS ME this is a textbook missing-virtualization problem. When you scroll a list of ten thousand items, your app should only ever have maybe twenty or thirty rows actually rendered into the DOM at any one time — the ones you can see, plus a small buffer above and below.

This is called list virtualization. It's been a solved problem in frontend engineering for over a decade.

A library that lags at a thousand items is one that's keeping all of them in the DOM. You only end up there if the list component was written assuming the lists wouldn't get that big — which makes sense if you came from a world of video search results.

FIGURE 03

Ten thousand rows, two ways.

Same scroll position. Same experience. Different cost.

FIG. 03 *Rendering ten thousand rows — all rendered vs. virtualized*



FIGURE NOTE. *Virtualization recycles rows as they leave the viewport, holding constant memory regardless of list length. Same scroll position, same UX, different cost.*

2.4 Cross-device handoff took eight years.

WHAT USERS HIT

Until recently, you could be listening on your laptop, switch to your phone, and YT Music would start the song over from the beginning. The position you were at didn't carry over. The queue didn't carry over. Spotify and Apple Music had this working from very early on. YT Music shipped position-syncing in early 2026.

WHAT THIS TELLS ME cross-device session continuity sounds like a feature, but it's actually a data model decision. To make it work, the app's idea of "what is currently playing" has to be a thing that lives in the cloud — a shared session object that any of your devices can read from and write to in real time.

If, instead, each device holds its own private state about what it's playing, then the best you can do is replicate the song reference itself. Not the position. Not the queue. Not the playback context.

YT Music shipped this eight years into the product. That isn't a UI bug. It's a foundational decision that didn't anticipate the experience users actually wanted — and required years to walk back.

2.5 Search that doesn't know what you want.

WHAT USERS HIT

You search for a song, you get songs mixed with music videos, sometimes with regular YouTube content bleeding in. The user is left to figure out which result was the one they wanted.

WHAT THIS TELLS ME a music search service should be its own thing — its own index, its own ranking, its own result schema. What looks more likely is that YT Music's search is going through the same retrieval system as YouTube proper, with a music filter applied. That works for unambiguous queries. It falls apart for anything generic. A purpose-built music search would know about covers vs originals, live versions, remixes, instrumental cuts — as first-class concepts, not variants of "different uploads of the same video."

2.6 The mini-player forgets.

WHAT USERS HIT

Lock the phone with music playing, unlock it, the app has restarted. The mini-player is gone, the queue is gone, you're back at home. Tablet users report this constantly.

WHAT THIS TELLS ME the audio playback service and the UI state aren't fully decoupled. In a music app, the foreground audio service has to be a robust, long-running thing that owns playback regardless of what the UI is doing. When the UI dying takes the audio with it, the service isn't as independent as it needs to be. That's not how Spotify works. That's not how Apple Music works. That, again, is how an app built around a video player works.

§ 03 — THE CLARIFYING MOMENT

The lyrics paywall.

If you only have time for one piece of evidence, this is the one.

In February 2026, YouTube Music started showing lyrics behind a Premium paywall for free-tier users.

A music app, designed as a music app, treats lyrics as part of the core experience. Spotify shows lyrics free. Apple Music shows lyrics free. The whole industry treats lyrics as something you obviously include in a music product, because lyrics are part of how people consume music.

YouTube Music decided lyrics were a Premium-tier feature. They put them behind the same kind of paywall that, on regular YouTube, sits in front of things like background playback and offline downloads — things that aren't part of the core video product but are convenience additions sold separately.

What this tells me is that internally, lyrics aren't being treated as a music thing. They're being treated as a YouTube-style monetizable add-on. **The mental model is YouTube's, not a music app's.**

You can debate whether removing landscape was an engineering shortcut. You can't really debate this one. Putting lyrics behind a paywall is what a company does when it sees music the way it sees video.

§ 04 — WHAT IT WOULD TAKE

Six foundations, not features.

A feature wishlist misses the point. The foundations are where the work is.

01 Now Playing as canonical view model.

One view, one state, one render. Every surface that opens it pulls from the same source.

02 Properly virtualized lists.

A one-quarter project for a motivated team. Not a re-architecture. A real fix.

03 Playback as a shared session entity.

Cloud-resident, not per-device. Cross-device handoff stops being a feature and becomes a property of the system.

04 Orientation as a first-class layout dimension.

Two layouts, not one stretched. Landscape Now Playing is the best canvas for synced lyrics — they walked away from it.

05 A music-native search service.

Decoupled from YouTube's video index. Covers, live, remixes, instrumentals as first-class concepts.

06 An independent playback service.

UI can die. Audio keeps going. Queue stays intact. This is how every other music app works.

§ 05 — CLOSING

The throne.

A discovery engine that's real, and unreplicable. A product around it that isn't.

YouTube Music has the best discovery engine in streaming. The product around that engine doesn't match. Fixing the product would let it win on merit.

The discovery engine is real and unreplicable. The product gaps are real too, and the reason they exist is not laziness or oversight. It's that YT Music has been a music product layered on a video platform from day one, and most of what's wrong follows from there.

Every visible symptom of the app points in the same direction. The only way YT Music wins on merit is by going back to those foundations and treating audio as the first-class product it is, instead of a tab on YouTube's house.

The throne is sitting empty. The argument for who gets it is going to be made one foundation at a time.

Colophon. An outside analysis based on publicly reported user issues and reasoned inference. No insider knowledge of YouTube Music's actual implementation.

Companion redesign and visual case study: **throne-empty.vercel.app**

Set in Source Serif 4 & IBM Plex Mono. Print edition · MMXXVI.